

Apostila de Pandas

Manipulação e Análise de Dados em Python

Teoria, exemplos práticos e principais usos

Sumário

1. Introdução ao Pandas
2. Instalação e Importação
3. Estruturas de dados: Series e DataFrame
4. Criando DataFrames
5. Leitura e escrita de arquivos (CSV, Excel, JSON)
6. Inspeccionando os dados
7. Seleção e indexação (loc, iloc)
8. Filtros e consultas (query)
9. Tratamento de valores ausentes
10. Tipos de dados e conversão
11. Criando e modificando colunas
12. Ordenação e ranking
13. Agrupamento (groupby) e agregações
14. Pivot table e crosstab
15. Concatenação, merge e join
16. Trabalhando com datas (datetime)
17. Strings e textos (.str)
18. Apply, map e funções vetorizadas
19. Estatísticas descritivas
20. Boas práticas e dicas finais

1. Introdução ao Pandas

O **Pandas** é a principal biblioteca Python para manipulação e análise de dados tabulares. Criado por Wes McKinney em 2008, ele oferece duas estruturas centrais: **Series** (vetor 1D rotulado) e **DataFrame** (tabela 2D rotulada em linhas e colunas), inspiradas nos data frames do R.

Por que usar Pandas?

- Leitura/escrita fácil de CSV, Excel, JSON, SQL, Parquet etc.
- Filtros, agrupamentos e junções (joins) expressivos
- Tratamento robusto de dados faltantes (NaN)
- Integração natural com NumPy, Matplotlib, scikit-learn

Nota: Pandas é base para qualquer trabalho de Data Science, BI e Machine Learning em Python.

2. Instalação e Importação

```
pip install pandas numpy openpyxl
```

Em seguida, o padrão é importar com o apelido **pd**:

```
import pandas as pd
import numpy as np
```

3. Estruturas de dados: Series e DataFrame

Series

Uma **Series** é um array unidimensional com um **índice** (rótulos). Pense nela como uma coluna de uma planilha.

```
import pandas as pd

s = pd.Series([10, 20, 30, 40], index=["a", "b", "c", "d"])
print(s)
print("Valor de 'c':", s["c"])

a    10
b    20
c    30
d    40
dtype: int64
Valor de 'c': 30
```

DataFrame

Um **DataFrame** é uma tabela: várias Series compartilhando o mesmo índice. Cada coluna pode ter um tipo diferente (int, float, string, datetime).

```
df = pd.DataFrame({
    "nome": ["Ana", "Bruno", "Carla"],
    "idade": [23, 35, 29],
    "cidade": ["SP", "RJ", "BH"],
})
print(df)
```

	nome	idade	cidade
0	Ana	23	SP
1	Bruno	35	RJ
2	Carla	29	BH

4. Criando DataFrames

Há várias formas de criar um DataFrame. As mais comuns:

A partir de um dicionário de listas

```
df = pd.DataFrame({
    "produto": ["A", "B", "C"],
    "preco":    [10.5, 20.0, 7.9],
    "estoque":  [100, 50, 200],
})
```

A partir de uma lista de dicionários

```
dados = [
    {"produto": "A", "preco": 10.5},
    {"produto": "B", "preco": 20.0},
]
df = pd.DataFrame(dados)
```

A partir de arrays NumPy

```
import numpy as np
arr = np.random.rand(4, 3)
df = pd.DataFrame(arr, columns=["x", "y", "z"])
```

5. Leitura e escrita de arquivos

Pandas suporta dezenas de formatos. Os mais usados no dia a dia:

```
# CSV
df = pd.read_csv("dados.csv")          # leitura
df.to_csv("saida.csv", index=False)    # escrita

# Excel (precisa de openpyxl)
df = pd.read_excel("planilha.xlsx", sheet_name="Vendas")
df.to_excel("saida.xlsx", index=False)

# JSON
df = pd.read_json("dados.json")
df.to_json("saida.json", orient="records", force_ascii=False)
```

Nota: Use **index=False** ao salvar para evitar que o índice numérico vire uma coluna no arquivo.

Parâmetros úteis em **read_csv**:

```
pd.read_csv(
    "vendas.csv",
    sep=";",      # separador (vírgula é o padrão)
    decimal=".",  # vírgula como decimal (padrão BR)
    encoding="utf-8", # ou "latin-1"
    parse_dates=["data"], # já converte para datetime
    na_values=["-", "NA"], # tratar como NaN
)
```

6. Inspeccionando os dados

Após carregar um DataFrame, sempre faça uma inspeção inicial:

```
df.head(5)          # primeiras 5 linhas
df.tail(5)          # últimas 5 linhas
df.shape            # (linhas, colunas)
df.columns          # nomes das colunas
df.dtypes           # tipo de cada coluna
df.info()           # resumo + memória + nulos
df.describe()       # estatísticas das colunas numéricas
df.isna().sum()     # quantos NaN por coluna
```

7. Seleção e indexação (loc e iloc)

Pandas oferece duas formas principais de selecionar dados:

- **loc** — seleciona por **rótulos** (nome de linha/coluna).
- **iloc** — seleciona por **posição inteira** (como em listas).

```
df = pd.DataFrame({
    "nome": ["Ana", "Bruno", "Carla", "Diego"],
    "idade": [23, 35, 29, 41],
    "salario": [3200, 5800, 4500, 7200],
}, index=["a", "b", "c", "d"])

df["nome"]          # uma coluna (Series)
df[["nome", "idade"]] # várias colunas (DataFrame)

df.loc["b"]         # linha pelo rótulo
df.loc["a":"c", ["nome", "idade"]] # fatia + colunas
df.iloc[0]          # primeira linha
df.iloc[0:2, 0:2]   # 2 primeiras linhas e colunas
```

Nota: Em **loc**, o fim do slice é **inclusivo** (ex.: 'a':'c' inclui 'c'). Em **iloc**, o fim é exclusivo, como em listas Python.

8. Filtros e consultas

Filtros são feitos com máscaras booleanas:

```
df[df["idade"] > 30]
df[(df["idade"] > 30) & (df["salario"] < 6000)]
df[df["nome"].isin(["Ana", "Carla"])]
df[~df["nome"].str.startswith("B")] # ~ inverte a máscara
```

A função **query** aceita uma expressão como string, ficando mais legível:

```
df.query("idade > 30 and salario < 6000")
df.query("nome in ['Ana', 'Carla'])")
```

9. Tratamento de valores ausentes (NaN)

Valores ausentes aparecem como **NaN**. Detectar e tratar é parte essencial da análise.

```
df.isna()                # True onde está faltando
df.isna().sum()          # total por coluna
df.dropna()              # remove linhas com qualquer NaN
df.dropna(subset=["idade"]) # só onde idade é NaN
df.fillna(0)             # preenche com 0
df["idade"].fillna(df["idade"].mean(), inplace=True) # média
df.fillna(method="ffill") # propaga o último valor válido
```

Nota: Antes de eliminar nulos, entenda o motivo: às vezes NaN tem significado (ex.: 'não respondeu').

10. Tipos de dados e conversão

```
df.dtypes
df["preco"] = df["preco"].astype(float)
df["id"]    = df["id"].astype("int64")
df["data"]  = pd.to_datetime(df["data"], format="%d/%m/%Y")
df["cat"]   = df["cat"].astype("category") # economiza memória
```

Nota: Use **category** para colunas com poucos valores distintos (sexo, UF, status) — reduz memória e acelera groupby.

11. Criando e modificando colunas

```
df["preco_com_imposto"] = df["preco"] * 1.10
df["faixa_idade"] = pd.cut(df["idade"], bins=[0,18,40,120],
                           labels=["menor", "adulto", "senior"])

# assign permite encadear de forma fluente:
df = df.assign(
    margem=lambda x: x["preco"] - x["custo"],
    margem_pct=lambda x: x["margem"] / x["preco"],
)

df = df.rename(columns={"preco": "valor"})
df = df.drop(columns=["coluna_x", "coluna_y"])
```

12. Ordenação e ranking

```
df.sort_values("salario", ascending=False)
df.sort_values(["cidade", "salario"], ascending=[True, False])
df["rank_salario"] = df["salario"].rank(ascending=False, method="dense")
```

13. Agrupamento (groupby) e agregações

O **groupby** segue o padrão *split-apply-combine*: divide os dados por uma chave, aplica uma função em cada grupo e combina o resultado.

```
vendas = pd.DataFrame({
    "loja":    ["A", "A", "B", "B", "C", "C"],
    "produto": ["X", "Y", "X", "Y", "X", "Y"],
    "qtd":     [10, 5, 8, 12, 7, 9],
    "preco":   [20, 30, 20, 30, 20, 30],
})
vendas["faturamento"] = vendas["qtd"] * vendas["preco"]

# Soma do faturamento por loja
vendas.groupby("loja")["faturamento"].sum()

# Várias agregações de uma vez
vendas.groupby("loja").agg(
    total=("faturamento", "sum"),
    media_qtd=("qtd", "mean"),
    n=("qtd", "count"),
)

# Por mais de uma chave
vendas.groupby(["loja", "produto"])["faturamento"].sum()
```

Nota: Funções comuns de agregação: **sum, mean, median, min, max, count, nunique, std, var, first, last**.

14. Pivot table e crosstab

pivot_table reorganiza dados longos em formato matricial (tipo tabela dinâmica do Excel):

```
vendas.pivot_table(
    index="loja",
    columns="produto",
    values="faturamento",
    aggfunc="sum",
    fill_value=0,
    margins=True,          # adiciona totais
)
```

crosstab é um atalho para contagem cruzada:

```
pd.crosstab(vendas["loja"], vendas["produto"])
```

15. Concatenação, merge e join

Concatenar (empilhar)

```
pd.concat([df1, df2], axis=0) # uma embaixo da outra
pd.concat([df1, df2], axis=1) # lado a lado
```

Merge (juntar por chave)

```
clientes = pd.DataFrame({"id": [1, 2, 3], "nome": ["A", "B", "C"]})
pedidos = pd.DataFrame({"id_cli": [1, 1, 2], "valor": [100, 200, 50]})

pd.merge(clientes, pedidos,
          left_on="id", right_on="id_cli",
          how="left") # left, right, inner, outer
```

Nota: Pense no **merge** como o **JOIN** do SQL. Use **how='inner'** para manter só as chaves comuns.

16. Trabalhando com datas (datetime)

```
df["data"] = pd.to_datetime(df["data"])

df["ano"]      = df["data"].dt.year
df["mes"]      = df["data"].dt.month
df["dia_sem"]  = df["data"].dt.day_name()

# Filtrar por intervalo
mask = (df["data"] >= "2025-01-01") & (df["data"] < "2025-02-01")
df[mask]

# Reamostragem (resample) — agrega por período
df.set_index("data").resample("M")["valor"].sum()  # mensal
df.set_index("data").resample("W")["valor"].mean() # semanal
```

17. Strings e textos (.str)

O acessor **.str** aplica operações vetorizadas em colunas de texto:

```
df["nome"].str.lower()
df["nome"].str.upper()
df["nome"].str.strip()          # tira espaços
df["nome"].str.contains("ana", case=False)
df["email"].str.split("@").str[1] # domínio do email
df["cpf"].str.replace(r"\D", "", regex=True) # só dígitos
```

18. apply, map e funções vetorizadas

Quando uma operação não existe pronta, use **apply** (em DataFrame/Series) ou **map** (em Series) com uma função:

```
df["faixa"] = df["idade"].apply(lambda x: "adulto" if x >= 18 else "menor")

# Em várias colunas
df[["a", "b"]].apply(np.sqrt)

# Linha a linha
df.apply(lambda row: row["preco"] * row["qtd"], axis=1)

# map: troca valores por dicionário
df["uf"] = df["uf"].map({"SP": "São Paulo", "RJ": "Rio de Janeiro"})
```

Nota: Prefira operações **vetorizadas** (ex.: `df['a']*df['b']`) — são muito mais rápidas que **apply** linha a linha.

19. Estatísticas descritivas

```
df.describe() # resumo de numéricas
df.describe(include="object") # resumo de texto
df["preco"].mean(), df["preco"].median(), df["preco"].std()
df["categoria"].value_counts() # contagem por categoria
df["categoria"].value_counts(normalize=True) # em %
df.corr(numeric_only=True) # matriz de correlação
```

20. Boas práticas e dicas finais

- **Sempre** inspecione com `head()`, `info()` e `describe()` antes de transformar.
- Evite loops for sobre linhas — use operações vetorizadas.
- Use `copy()` ao criar um subconjunto que será modificado (evita *SettingWithCopyWarning*).

- Mantenha um **pipeline** claro: leitura → limpeza → transformação → análise.
- Documente suas etapas; um notebook bem comentado vale ouro.
- Combine com **Matplotlib/Seaborn** para visualizar e com **scikit-learn** para modelar.

Próximos passos: pratique com datasets reais (Kaggle, dados.gov.br), explore **Polars** (alternativa rápida ao Pandas) e estude **SQL** — os conceitos de groupby e join se reforçam mutuamente.

— Fim da apostila —